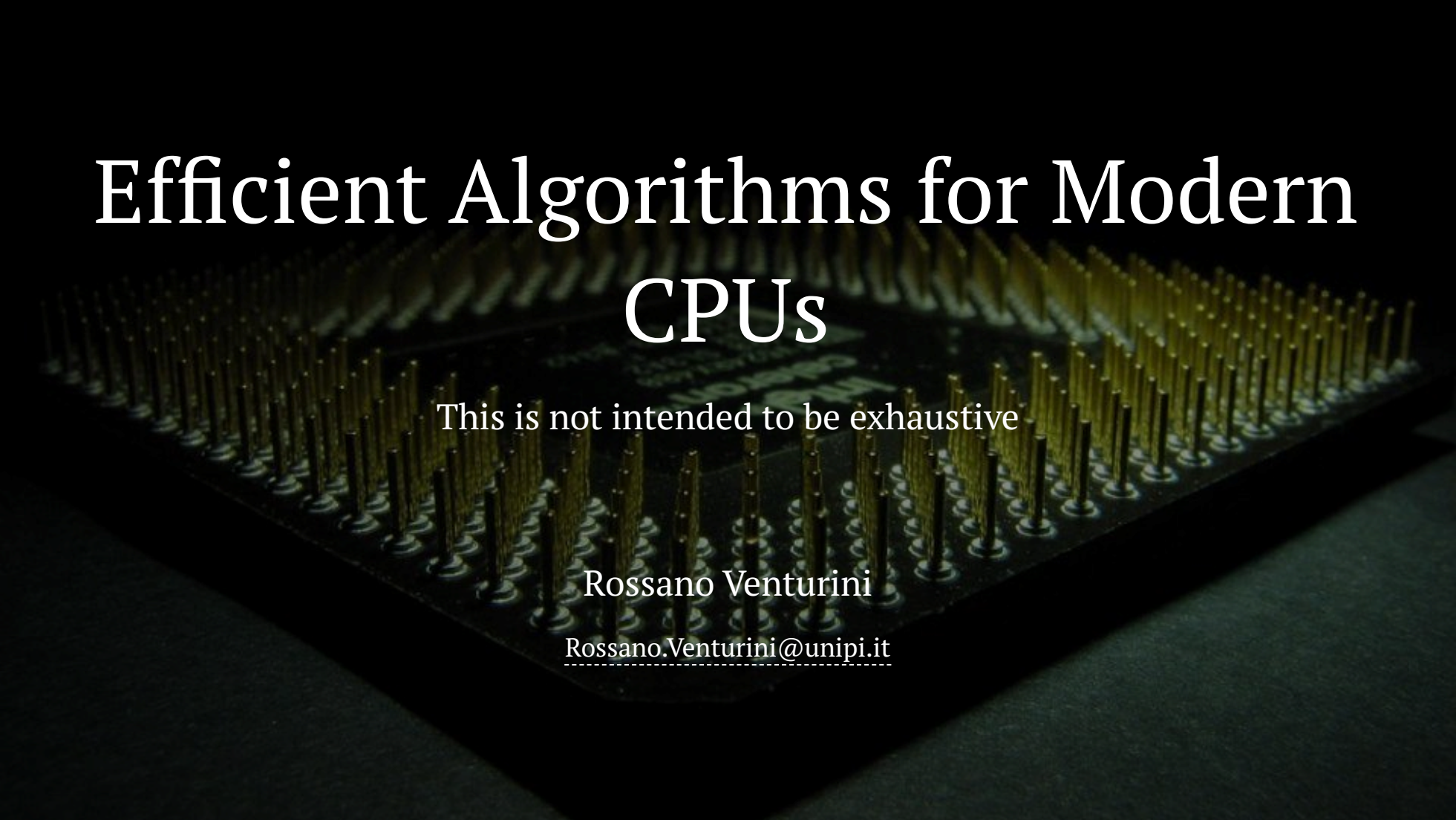




Efficient Algorithms for Modern CPUs

This is not intended to be exhaustive

Rossano Venturini

Rossano.Venturini@unipi.it

Why Efficiency Matters in Information Retrieval

- **User Experience**

- Faster responses lead to *better user satisfaction*
- More effective algorithms within the same time budget
- *Impact*: Delays or irrelevant results can cause users to switch to competitors

- **Scalability**

- *Large Data Volumes*: IR systems handle *billions* of documents/items and queries
- *Efficient Algorithms*: Needed to scale retrieval across *global infrastructures*
- *Cost Reduction*: Efficiency reduces *computational and storage costs*
 - both for computing and cooling systems

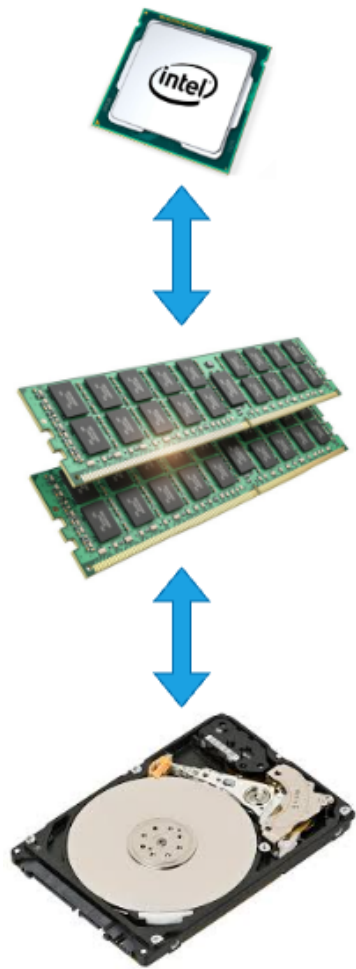
- **Environmental Sustainability**

- *Energy Consumption*: Data centers consume vast energy; efficiency reduces *carbon footprint*

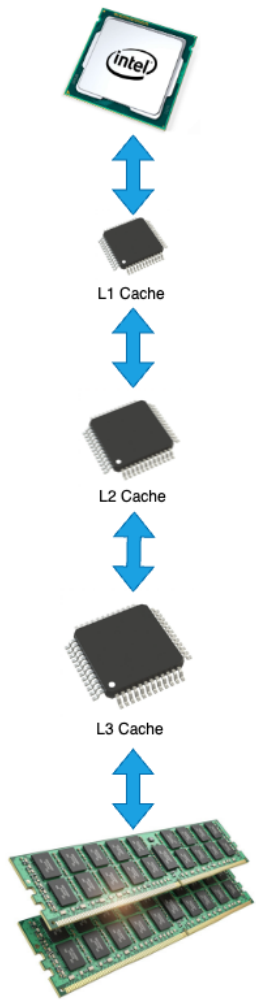
Data Centers



Google

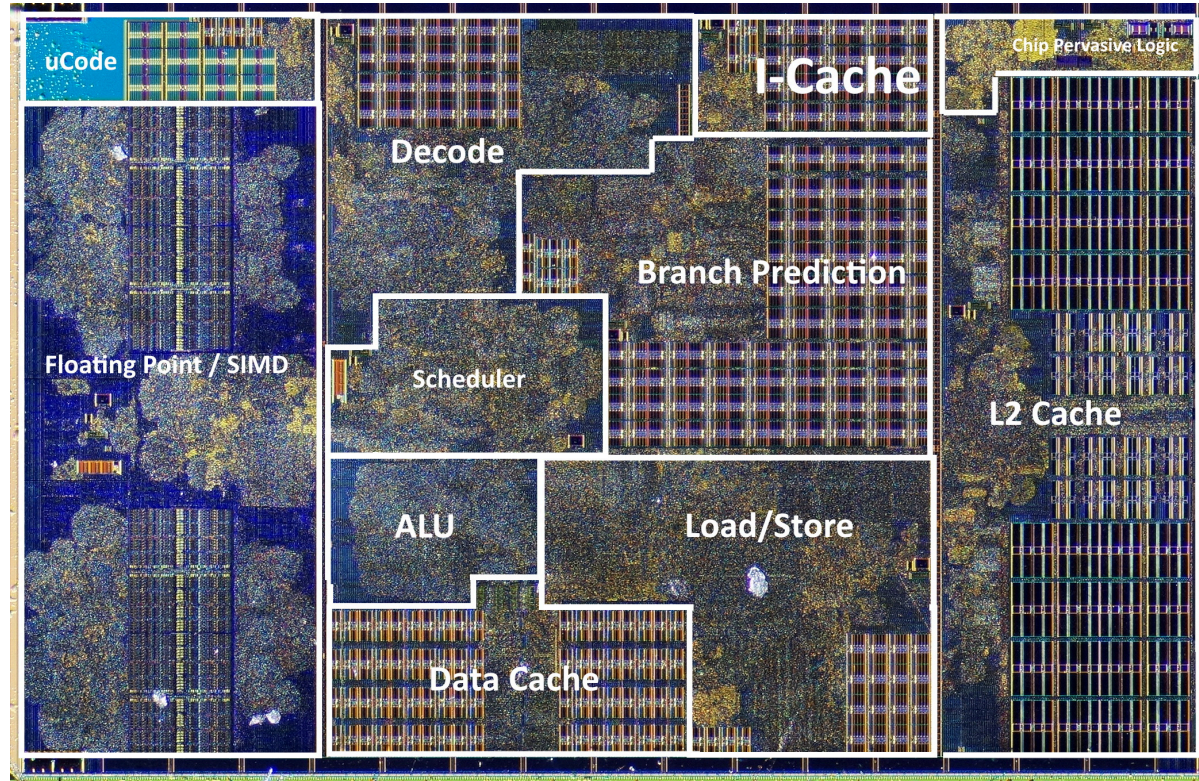


	Latency (ns)	Size
Instruction	0.5–5	-
RAM	80	64 GBs
Disk	500,000	8 TBs



	Latency (ns)	Size
Instruction	0.5–5	-
L1I Cache	0.5	32 KBs
L1D Cache	0.5	32 KBs
L2 Cache	1.5	256 KBs
L3 Cache	5	16 MBs
RAM	80	64 GBs
Disk	500,000	8 TBs

L1 and L2 Caches are in the CPU



Memory Hierarchy

Your programs would exploit CPU memory hierarchy if they have at least one of two fundamental properties

- **Temporal locality:** when a given memory location is accessed, it is likely that the same location will be accessed again in the near future
 - *Replacement policy* favors least recently used data when cache is full
- **Spatial locality:** when a given memory location is accessed, it is likely that nearby locations will be accessed in the near future.
 - Important because when access a location, a larger chunk of memory (a **cache line** of 64 bytes) is fetched
 - **Hardware prefetchers** observe the behavior of a running application and initiate prefetching on repetitive patterns of cache misses.

What's the fastest?

```
let pi: Vec<usize> = random_permutation_with_one_cycle( SIZE );
```

```
pub fn sum(v: &[usize]) -> usize {  
    let mut sum = 0;  
  
    for &value in v.iter() {  
        sum += value;  
    }  
  
    sum  
}
```

```
pub fn sum_by_jumping(v: &[usize]) -> usize {  
    let mut sum = 0;  
  
    let mut k = 0;  
    for _ in 0..v.len() {  
        sum += v[k];  
        k = v[k];  
    }  
  
    sum  
}
```

What's the fastest?

```
// v has 256 values (2 KiBs) (i.e., fits in L1 cache)
sum():           Time per value: 0.1 ns avg
sum_by_jumping(): Time per value: 1.3 ns avg
```

```
// v has 8192 values (64 KiBs) (i.e., fits in L2 cache)
sum():           Time per value: 0.1 ns avg
sum_by_jumping(): Time per value: 2.5 ns avg
```

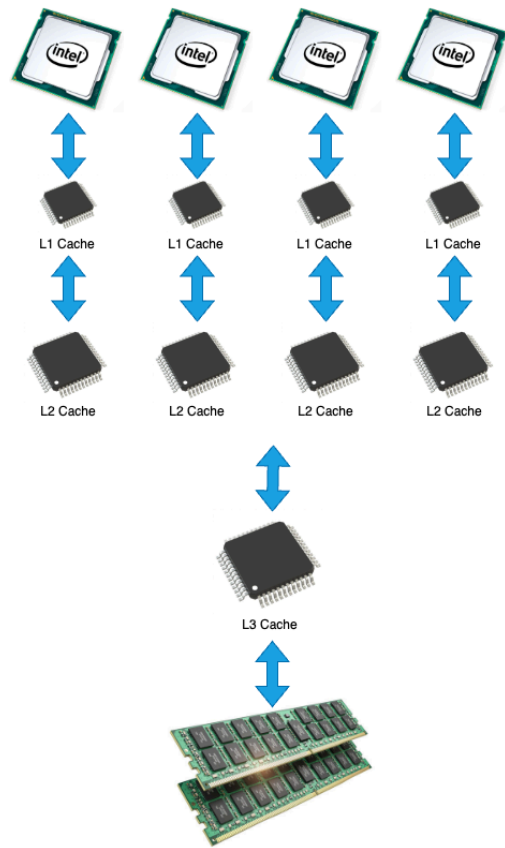
```
// v has 262144 values (2 MiBs) (i.e., fits in L3 cache)
sum():           Time per value: 0.2 ns avg
sum_by_jumping(): Time per value: 13.4 ns avg
```

```
// v has 16777216 values (128 MiBs) (i.e., fits in RAM cache)
sum():           Time per value: 0.4 ns avg
sum_by_jumping(): Time per value: 84.9 ns avg
```

Challenge 1: Write a function (similar to `sum()` , but using different operations) where you do not observe any slowdown as the size increases

Challenge 2: write a function (similar to `sum_by_jumping()`) that jumps over the cache lines summing all the values in the cache line

Parallel Computing



Let's move within a single CPU

Speeding up a CPU with different forms of parallelism: *Do more things at the same time*

- **Pipelining:** Overlapping the execution of instructions so that different parts of the CPU are kept busy
- **Superscalar processors:** Adding multiple execution units to process independent operations simultaneously
- **SIMD:** Adding instructions capable of executing the same operation on more data at the same time

Pipelining

Pipelining is the foundational technique used to make CPUs fast wherein multiple instructions are overlapped during their execution.

The stages operate in parallel, working on different parts of different instructions.

A 5-stage pipeling may be like this:

- **fetching** a chunk of machine code from memory
- **decoding** it and splitting it into (micro)instructions
- **executing** these instructions, which may involve doing some **memory** operations
- **writing** the results back into registers

Pipelining

- The time required to move an instruction from one stage to the next defines the basic machine cycle or clock for the CPU
 - The value chosen for the clock for a given pipeline is defined by the slowest stage of the pipeline
- Pipeline are made "*deeper*", a given step can be implemented with simpler circuitry, which may let the processor clock run faster
- Modern high-performance CPUs have multiple pipeline stages, often ranging from 10 to 20 or more
 - Latency to execute something simple like `add ing` may be 10-20 CPU cycles
 - The **latency** for any given instruction is the total time through all the stages of the pipeline
 - The **throughput** of a pipelined CPU is defined as the number of instructions that complete and exit the pipeline per unit of time

What's the fastest?

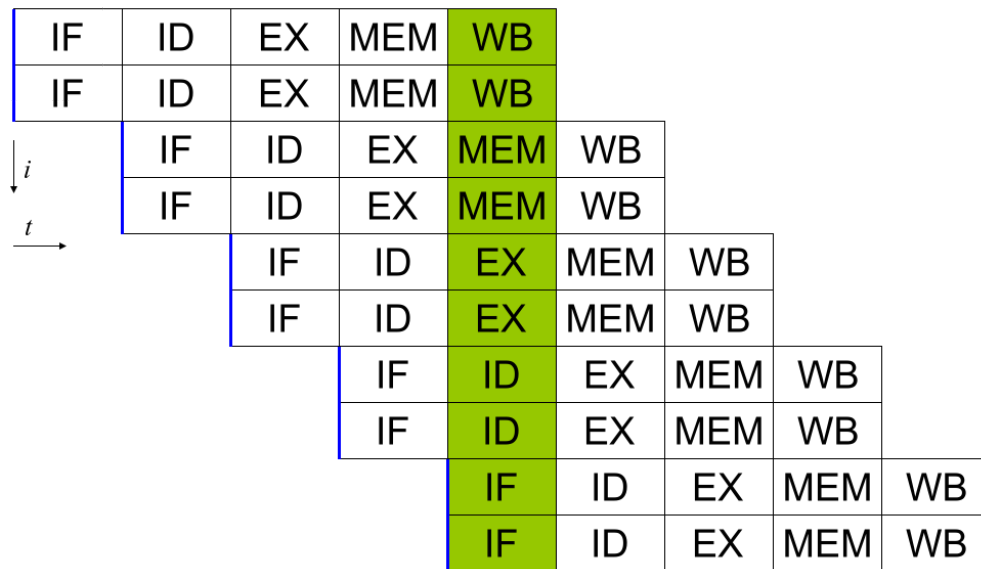
```
pub fn do_stuff(v: &[u64]) -> u64 {  
    let mut prod = 1;  
  
    for &value in v.iter() {  
        prod *= value;  
    }  
    prod  
}
```

```
pub fn do_more_stuff(v: &[u64]) -> (u64, u64) {  
    let mut prod = 1;  
    let mut sum = 0;  
  
    for &value in v.iter() {  
        prod *= value;  
        sum += value;  
    }  
    (prod, sum)  
}
```

do_stuff():	Time per iteration: 0.4 ns avg
do_more_stuff():	Time per iteration: 0.4 ns avg

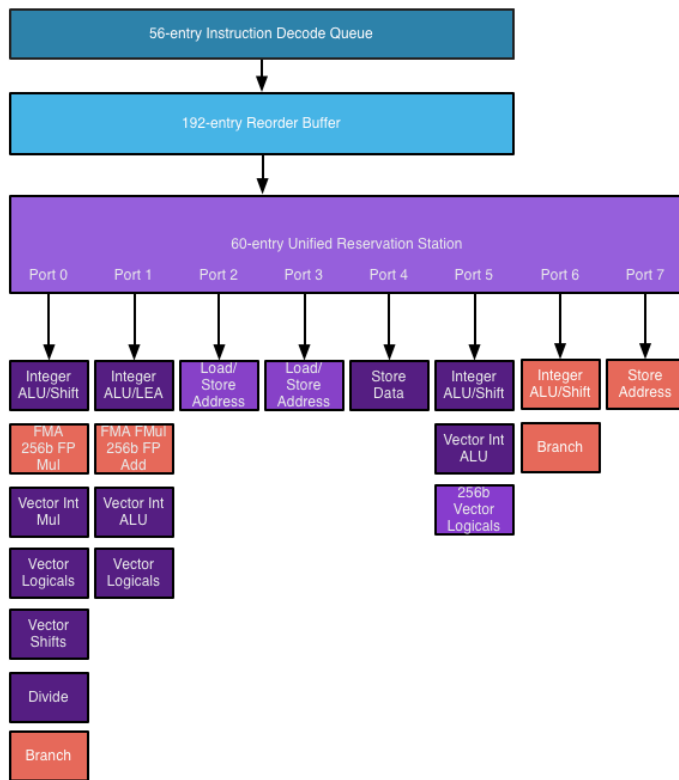
Superscalar Processing

- The ability to process more than one instruction per clock cycle within a single CPU core.
 - Achieved by simultaneously dispatching multiple instructions to different execution units on the core
 - Each core has multiple specialized Arithmetic Logic Units (ALUs)
 - Also known as *instruction-level parallelism*



Superscalar Processing

Intel Haswell Execution Engine



What about this?

```
pub fn do_more_stuff_2(v: &[u64]) -> (u64, u64, u64) {  
    let mut prod = 1;  
    let mut sum = 0;  
    let mut xor = 0;  
  
    for &value in v.iter() {  
        prod *= value;  
        sum += value;  
        xor ^= value;  
    }  
  
    (prod, sum, xor)  
}
```

do_stuff():	Time per iteration: 0.4 ns avg
do_more_stuff():	Time per iteration: 0.4 ns avg
do_more_stuff_2():	Time per iteration: 0.4 ns avg

and this?

```
#[inline]
pub fn do_more_stuff_3(v: &[u64]) -> (u64, u64, u64, u64) {
    let mut prod = 1;
    let mut sum = 0;
    let mut xor = 0;
    let mut or = 0;

    for &value in v.iter() {
        prod *= value;
        sum += value;
        xor ^= value;
        or |= value;
    }

    (prod, sum, xor, or)
}
```

do_stuff():	Time per iteration: 0.4 ns avg
do_more_stuff():	Time per iteration: 0.4 ns avg
do_more_stuff_2():	Time per iteration: 0.4 ns avg
do_more_stuff_3():	Time per iteration: 0.5 ns avg

Pipeline Hazards

Pipeline hazards are situations when the next instruction cannot execute on the following clock cycle.

There are multiple ways this may happen:

- **Structural Hazard:** more instructions need the same part of CPU (e.g., an execution unit).
 - They are fundamental bottlenecks on performance and can't be avoided

and this?

```
pub fn do_more_stuff_4(v: &[u64]) -> (u64, u64, u64) {  
    let mut prod = 1;  
    let mut sum = 0;  
    let mut xor = 0;  
  
    for &value in v.iter() {  
        sum += value;  
        prod *= sum;  
        xor ^= prod;  
        sum += xor;  
    }  
  
    (prod, sum, xor)  
}
```

do_stuff():	Time per iteration: 0.4 ns avg
do_more_stuff():	Time per iteration: 0.4 ns avg
do_more_stuff_2():	Time per iteration: 0.4 ns avg
do_more_stuff_3():	Time per iteration: 0.5 ns avg
do_more_stuff_4():	Time per iteration: 1.7 ns avg

Pipeline Hazards

Pipeline hazards are situations when the next instruction cannot execute on the following clock cycle.

There are multiple ways this may happen:

- **Structural Hazard:** more instructions need the same part of CPU (e.g., an execution unit).
 - They are fundamental bottlenecks on performance and can't be avoided
- **Data Hazard:** need to wait for an operand to be computed from some previous step.
 - Solution: **restructure computations** so that the critical path is shorter.

What about this?

```
pub fn do_more_stuff_5(v: &[u64]) -> (u64, u64, u64) {  
    let mut prod = 1;  
    let mut sum = 0;  
    let mut xor = 0;  
  
    for &value in v.iter() { // values are random values in [0, 99]  
        if value > 50 {  
            sum += value;  
            prod *= sum;  
            xor ^= prod;  
            sum += xor;  
        }  
    }  
  
    (prod, sum, xor)  
}
```

do_stuff():	Time per iteration: 0.4 ns avg
do_more_stuff():	Time per iteration: 0.4 ns avg
do_more_stuff_2():	Time per iteration: 0.4 ns avg
do_more_stuff_3():	Time per iteration: 0.5 ns avg
do_more_stuff_4():	Time per iteration: 1.7 ns avg
do_more_stuff_5():	Time per iteration: 3.5 ns avg

Pipeline Hazards

Pipeline hazards are situations when the next instruction cannot execute on the following clock cycle.

There are multiple ways this may happen:

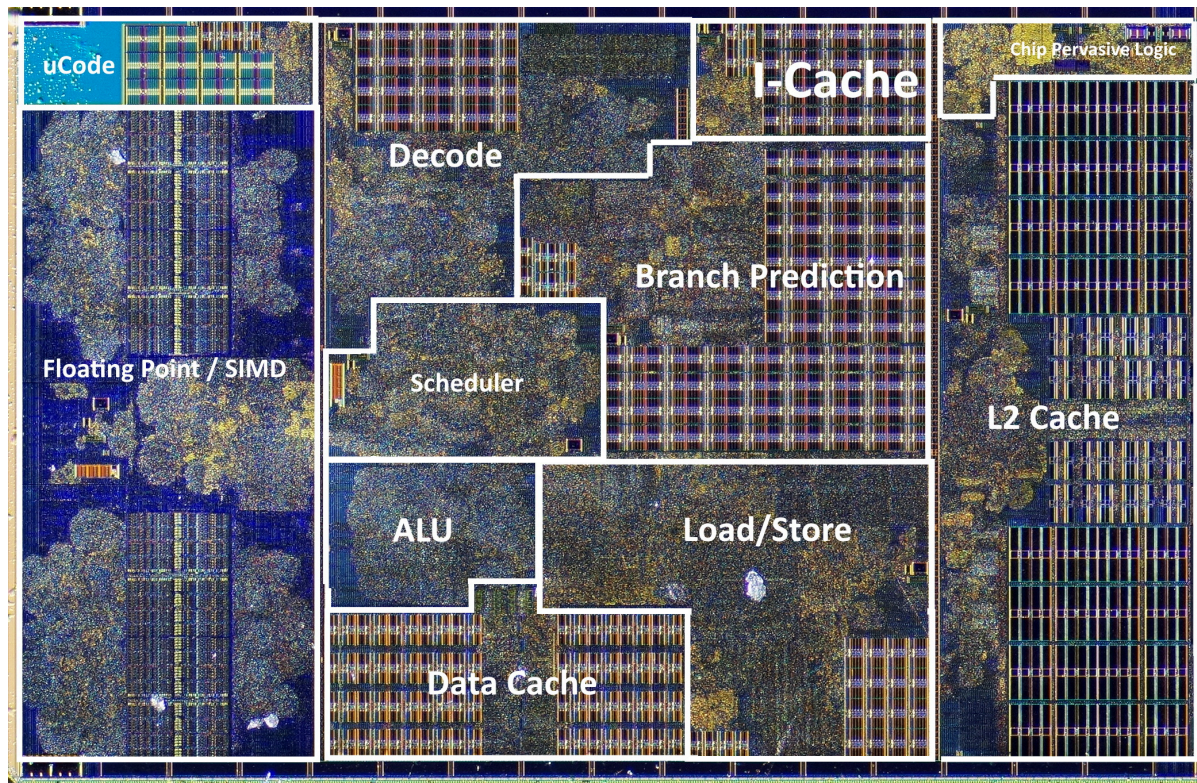
- A **structural** hazard happens when two or more instructions need the same part of CPU (e.g., an execution unit).
 - They are fundamental bottlenecks on performance and can't be avoided
- **Data Hazard**: need to wait for an operand to be computed from some previous step.
 - Solution: **restructure computations** so that the critical path is shorter.
- **Control Hazard**: the CPU can't tell which instructions it needs to execute next.
 - CPU has to choose one of the two branches.
 - if wrong, it has to flush the entire pipeline and start over, wasting a whole 15-20 cycles
 - Solution: either **remove branches completely** or **make them predictable**

Same code but `v` is sorted!

```
pub fn do_more_stuff_5(v: &[u64]) -> (u64, u64, u64) {  
    let mut prod = 1;  
    let mut sum = 0;  
    let mut xor = 0;  
  
    for &value in v.iter() { // sorted random values in [0, 99]  
        if value > 50 {  
            sum += value;  
            prod *= sum;  
            xor ^= prod;  
            sum += xor;  
        }  
    }  
  
    (prod, sum, xor)  
}
```

do_more_stuff_4():	Time per iteration: 1.7 ns avg (no if)
do_more_stuff_5():	Time per iteration: 3.5 ns avg (random)
do_more_stuff_5():	Time per iteration: 1.2 ns avg (sorted)

Branch Prediction



Branch Prediction

- A **branch predictor** is a part of the CPU that tries to guess which way a branch will go before this is known definitively
 - Most prediction algorithms are based on previous outcomes of the branch
- A **branch misprediction** costs is equal to the number of stages in the pipeline from the fetch stage to the execute stage (15 – 20 cycles)
- Many typical patterns are recognized
 - Almost always taken/not taken (e.g., condition in a loop)
 - A branch behaves consistently for a few iterations and then changes (e.g., alternately taken/not taken)
 - Many others with more sophisticated predictors

Values in `v` are alternated!

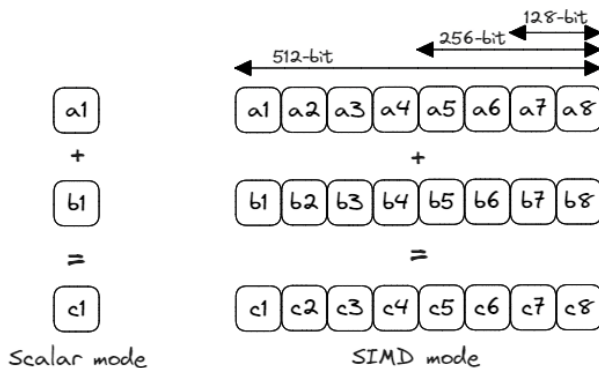
```
pub fn do_more_stuff_5(v: &[u64]) -> (u64, u64, u64) {  
    let mut prod = 1;  
    let mut sum = 0;  
    let mut xor = 0;  
  
    for &value in v.iter() {  
        if value > 50 {  
            sum += value;  
            prod *= sum;  
            xor ^= prod;  
            sum += xor;  
        }  
    }  
  
    (prod, sum, xor)  
}
```

do_more_stuff_4():	Time per iteration: 1.7 ns avg (no if)
do_more_stuff_5():	Time per iteration: 3.5 ns avg (random)
do_more_stuff_5():	Time per iteration: 1.2 ns avg (sorted)
do_more_stuff_5():	Time per iteration: 1.1 ns avg (alternated)

Challenge: Try to find other patterns your CPU can recognize.

SIMD: Single Instruction, Multiple Data

- Modern CPUs support extensions to the x86 instruction called “SIMD extensions”
 - MMX, SSE, SSE2, SSE3, SSE4.1, SSE4.2, AVX, AVX2, AVX512
- Instructions that operate on special registers holding 128, 256, or even 512 bits
- SIMD instructions divide the data in registers into blocks of 8, 16, 32, or 64 bits
- They operate a “*single instruction, multiple data*” (SIMD) in parallel
- This yields a proportional increase in performance



Using SIMD in Programming

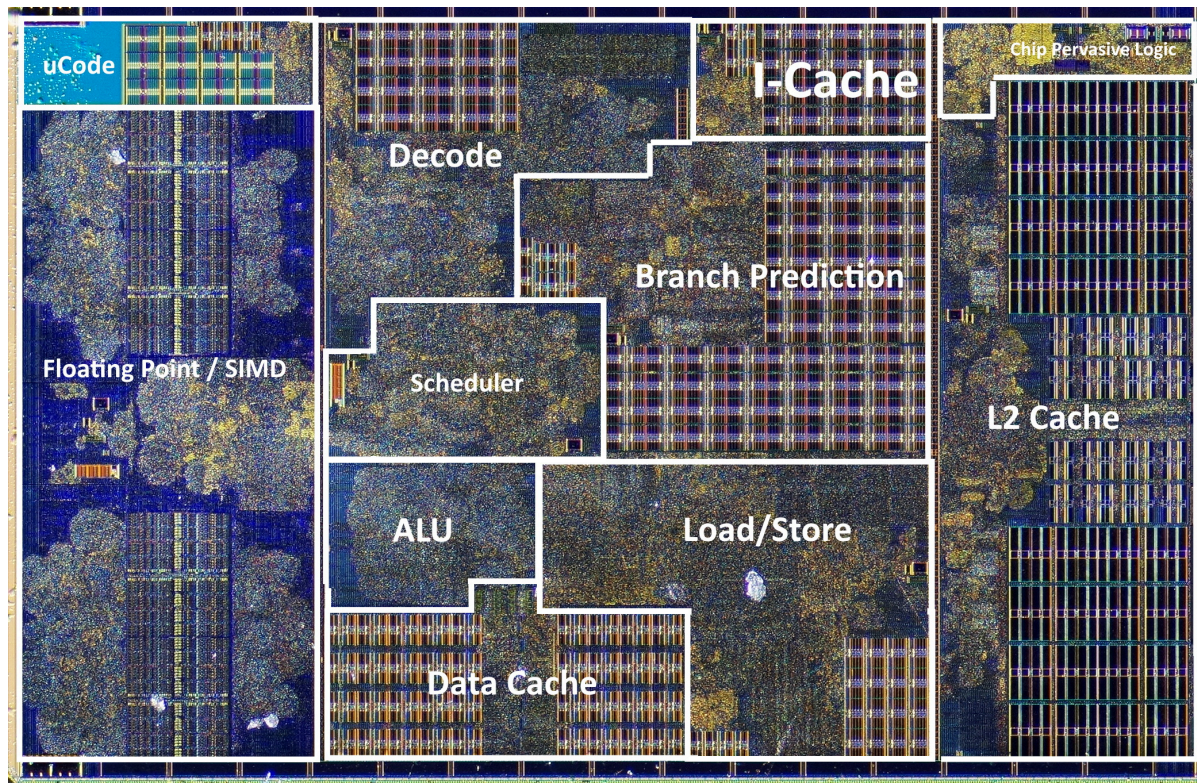
- **Auto-vectorization**

- Compiler automatically converts scalar operations into SIMD instructions
- *Advantages*
 - No manual code changes required
- *Drawbacks*
 - Only works with simple loops or operations that the compiler can optimize

- **Intrinsics**

- Developers write specific instructions to take full advantage of SIMD
- *Advantages*
 - Higher control over the hardware and Can optimize complex operations
- *Drawbacks*
 - The approach requires detailed hardware knowledge and the code is platform-specific

SIMD



Do you remember?

```
pub fn sum(v: &[usize]) -> usize {  
    let mut sum = 0;  
  
    for &value in v.iter() {  
        sum += value;  
    }  
    sum  
}
```

```
pub fn sum_by_jumping(v: &[usize]) -> usize {  
    let mut sum = 0;  
  
    let mut k = 0;  
    for _ in 0..v.len() {  
        sum += v[k];  
        k = v[k];  
    }  
    sum  
}
```

```
// v has 256 values (2 KiBs) (i.e., fits in L1 cache)  
sum():           Time per value: 0.1 ns avg  
sum_by_jumping(): Time per value: 1.3 ns avg
```

Why?

- Compiler Explorer `sum()`
 - Performs four sums at a time
 - A reduce at the end
- Compiler Explorer `sum_by_jumping()`
 - Performs a add at a time
- If data fits in L1 cache, then CPU bound

References

- Sections 3.1, 3.2, 3.3, 3.4, and 3.6 in "CPU Microarchitecture: Performance Analysis and Tuning on Modern CPUs", Denis Bakhvalov, 2024 ([link](#))
- Sections 3.1, 3.2, 9.1, 9.2, 9.3, 9.6, 10.1, and 10.6 in *Algorithms for Modern Hardware*, Sergey Slotin ([link](#))